

BSM-308 System Programming

Sakarya University-Computer Engineering

Contents

- [Fields](#) - a library to simplify input processing
- [Jvals](#) - a generic data type
- [Dllists](#) - a library for doubly-linked lists
- [Red-Black Trees](#) - a library for red-black trees

• <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Jval/index.html>

• <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Dllists/index.html>

• <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/JRB/index.html>

Jvals , Fields, Dllists , Red-Black Trees

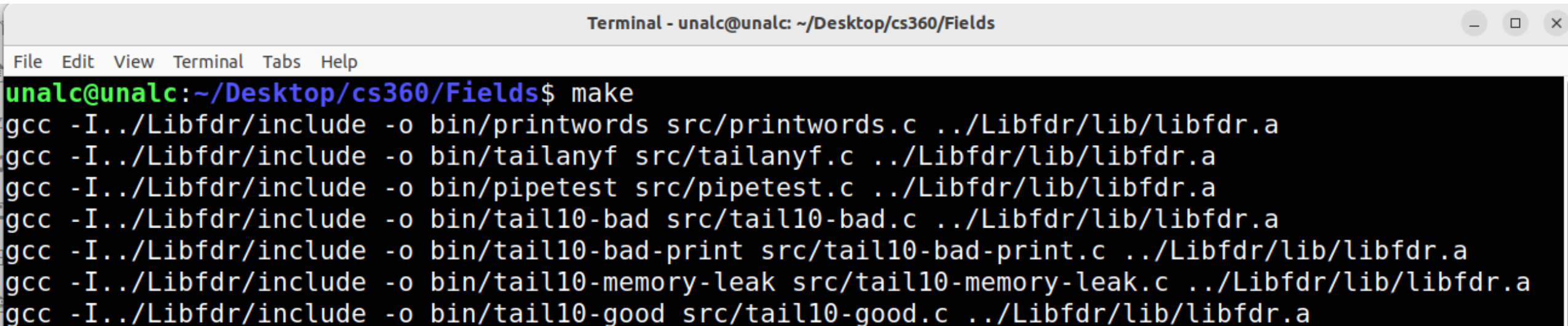
Compilation and Linking

To compile your code, you need to use the compiler flag **-I/home/jplank/cs360/include**. For example, if you have included `fields.h` and `dllist.h` in your program **lab1.c**, then your first compilation step is:

```
gcc -g -I/home/jplank/cs360/include -c lab1.c
```

That will make **lab1.o**. Now, when you go to link **lab1.o** into an executable, you need to link with **/home/jplank/cs360/objs/libfdr.a**:

```
gcc -g -o lab1 lab1.o /home/jplank/cs360/objs/libfdr.a
```

A terminal window titled "Terminal - unalc@unalc: ~/Desktop/cs360/Fields" is shown. The window has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The terminal content shows the user running the "make" command, which triggers a series of gcc compilation commands. Each command compiles a source file from the "src" directory into an object file in the "bin" directory, linking against the library "../Libfdr/lib/libfdr.a". The source files being compiled are printwords.c, tailanyf.c, pipetest.c, tail10-bad.c, tail10-bad-print.c, tail10-memory-leak.c, and tail10-good.c.

```
Terminal - unalc@unalc: ~/Desktop/cs360/Fields
File Edit View Terminal Tabs Help
unalc@unalc:~/Desktop/cs360/Fields$ make
gcc -I../Libfdr/include -o bin/printwords src/printwords.c ../Libfdr/lib/libfdr.a
gcc -I../Libfdr/include -o bin/tailanyf src/tailanyf.c ../Libfdr/lib/libfdr.a
gcc -I../Libfdr/include -o bin/pipetest src/pipetest.c ../Libfdr/lib/libfdr.a
gcc -I../Libfdr/include -o bin/tail10-bad src/tail10-bad.c ../Libfdr/lib/libfdr.a
gcc -I../Libfdr/include -o bin/tail10-bad-print src/tail10-bad-print.c ../Libfdr/lib/libfdr.a
gcc -I../Libfdr/include -o bin/tail10-memory-leak src/tail10-memory-leak.c ../Libfdr/lib/libfdr.a
gcc -I../Libfdr/include -o bin/tail10-good src/tail10-good.c ../Libfdr/lib/libfdr.a
```

jval.h

- ❑ If you don't know, a union is like a struct , but you can only use one of the fields at a time.
- ❑ In other words, if you have a Jval named `jv` , you can set `jv.i` as an integer and `jv.d` as a double and use,
- ❑ but you cannot use both at the same time.

```
typedef union {  
    int i;  
    long l;  
    float f;  
    double d;  
    void *v;  
    char *s;  
    char c;  
    unsigned char uc;  
    short sh;  
    unsigned short ush;  
    unsigned int ui;  
    int iarray[2];  
    float farray[2];  
    char carray[8];  
    unsigned char uarray[8];  
} Jval;
```

Jval.c

```
/* Libraries for fields, doubly-linked lists and red-black trees.  
   Copyright (C) 2018 James S. Plank */
```

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <string.h>  
#include "jval.h"
```

```
Jval JNULL;
```

```
Jval new_jval_i(int i) {  
    Jval j;  
    j.i = i;  
    return j;  
}
```

```
Jval new_jval_l(long l) {  
    Jval j;  
    j.l = l;  
    return j;  
}
```

src / simple_jval.c

```
/* This program shows the very simple use of a jval. */

#include <stdio.h>
#include <stdlib.h>
#include "jval.h"

int main()
{
    Jval jv;

    /* Set and use as an integer. */

    jv.i = 10;
    printf("Using the jval as an integer: %d\n", jv.i);

    /* Set and use as a double. */

    jv.d = 5.6;
    printf("Using the jval as a double: %.2lf\n", jv.d);

    /* Set and use as a char *. */

    jv.s = "Fred";
    printf("Using the jval as a char *: %s\n", jv.s);

    /* Since I've set it as a (char *), I shouldn't use it as a double. */

    printf("If I now print the jval as a double, I get garbage: %lg\n", jv.d);
    return 0;
}
```

UNIX> **bin/simple_jval**

Using the jval as an integer: 10

Using the jval as a double: 5.60

Using the jval as a char *: Fred

If I now print the jval as a double, I get garbage: 2.13063e-314

UNIX>

specifier character

specifier	output	examples
d or i	Signed decimal integer	392
u	Unsigned decimal integer	7235
he	Unsigned octal	610
x	Unsigned hexadecimal integer	7fa
X	Unsigned hexadecimal integer (uppercase)	7FA
f	Decimal floating point, lowercase	392.65
F	Decimal floating point, uppercase	392.65
to	Scientific notation (mantissa/exponent), lowercase	3.9265e+2
TO	Scientific notation (mantissa/exponent), uppercase	3.9265E+2
g	Use the shortest representation: %e or %f	392.65
g	Use the shortest representation: %E or %F	392.65
a	Hexadecimal floating point, lowercase	-0xc.90fep-2
A.	Hexadecimal floating point, uppercase	-0XC.90FEP-2
c	character	a
s	String of characters	samples
p	Pointer address	b8000000
n	Nothing printed. The corresponding argument must be a pointer to a signed int. The number of characters written so far is stored in the pointed location.	
%	A % followed by another % character will write a single % to the stream.	%

Constructor methods

You can of course, create and use a **Jval** by simply declaring and using it. For example:

```
Jval j;  
  
j.i = 4;
```

You can freely pass **Jval**'s to and from procedure calls. A **Jval** can be the return value of a procedure call.

Jval.h defines a whole bunch of prototypes for "constructor functions":

```
extern Jval new_jval_i(int);  
extern Jval new_jval_f(float);  
extern Jval new_jval_d(double);  
extern Jval new_jval_v(void *);  
extern Jval new_jval_s(char *);
```

These return **Jval**'s to you when you give them arguments of a specific type. For example, if you want to initialize a **Jval** so that it is in integer whose value is 4, you can do it as shown above, or you can do:

```
Jval j;  
  
j = new_jval_i(4);
```

Now **j.i** will be the integer 4. This is convenient when you are using a procedure that takes a **Jval** as a parameter, because you can call the procedure and never actually declare the **Jval**. Here's an example, in [src/jval_ex2.c](#):

src /jval_ex2.c :

```
/* A program that demonstrates the "constructor" functions, and how they help
   when you are using a procedure that takes a jval as a parameter. */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "fields.h"
#include "jval.h"

/* Print a jval as an integer, double or string. */

void print_jval(Jval jv, char how)
{
    switch (how) {
        case 'i': printf("%d\n", jv.i); break;
        case 'd': printf("%lg\n", jv.d); break;
        case 's': printf("%s\n", jv.s); break;
        default: fprintf(stderr, "Bad 'how' variable in print_jval: %c\n", how);
                    exit(1);
    }
}

/* Call print_jval.  Note that I never have to declare a jval, because I'm using
   the constructor functions. */
int main()
{
    print_jval(new_jval_i(10), 'i');
    print_jval(new_jval_d(5.6), 'd');
    print_jval(new_jval_s("Fred"), 's');
    return 0;
}
```

```
UNIX> bin/jval_ex2
10
5.6
Fred
UNIX>
```

Misuses

- ❑ Jval type is to make general-purpose data structures such as dllists and red-black trees as flexible and efficient as possible.
- ❑ jvals in your code for any other reason.
- ❑ Specifically, you don't say use Jval instead of int in your code just because it works .
- ❑ This makes your code unreadable, and unreadability is largely bad.
- ❑ Here is an example of bad code to average all integers in standard input

src / badavg.c

```
/* A program that abuses jvals. It averages integers on standard input. */

#include <stdio.h>
#include <stdlib.h>
#include "jval.h"

int main()
{
    Jval total;      /* You'll note, no types other than Jvals. */
    Jval j;
    Jval n;

    n.i = 0;          /* n and j are always treated as ints. They should be ints. */
    total.i = 0;       /* Total starts as an int */

    while (scanf("%d", &(j.i)) == 1) {
        total.i += j.i;
        n.i++;
    }

    /* And then total is used as a double. You should simply use a double.... */

    total.d = ((double) total.i) / ((double) n.i);
    printf("Average = %lf\n", total.d);
    return 0;
}
```

src / goodavg.c

```
/* The proper version of this program just uses doubles for total and n. It's not
   like there are ints that can't be stored in a double, and you're using floating
   point division, so you need the doubles. */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    double total;
    double n;
    int j;

    n = 0;
    total = 0;

    while (scanf("%d", &j) == 1) {
        total += j;
        n++;
    }

    if (n == 0) exit(1);
    printf("Average = %lf\n", total/n);

    return 0;
}
```

Accessor functions

- An access function is a function rather than accessing a field . It allows you to get the desired value from a Jval by calling it .

```
extern int    jval_i(Jval);
extern long   jval_l(Jval);
extern float  jval_f(Jval);
extern double jval_d(Jval);
extern void   *jval_v(Jval);
extern char   *jval_s(Jval);
extern char   jval_c(Jval);
...
```

So, for example, calling **jval_i(j)** is the same as using **j.i**.

Doubly Linked lists

- ❑ Doubly linked lists (" dllists ") are extremely useful in C because they allow you to store data when you don't know how much data you have.
- ❑ In a doubly linked list, each node has two pointers - one that points to the next node in the list and one that points to the previous node. In this way;
 - ❑ You can move lists forward and backward.
 - ❑ You can easily add it anywhere in a list.
 - ❑ You can delete nodes very easily.

```
typedef struct dllist {  
    struct dllist *flink;  
    struct dllist *blink;  
    Jval val;  
} *Dllist;
```

dllist.c

```
/* Libraries for fields, doubly-linked lists and red-black trees.
   Copyright (C) 2018 James S. Plank */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "dllist.h"

/*-----*
 * PROCEDURES FOR MANIPULATING DOUBLY LINKED LISTS
 * Each list contains a sentinal node, so that
 * the first item in list l is l->flink.  If l is
 * empty, then l->flink = l->blink = l.
 *-----*/

Dllist new_dllist()
{
    Dllist d;

    d = (Dllist) malloc (sizeof(struct dllist));
    d->flink = d;
    d->blink = d;
    return d;
}
```

dllist.h

```
/* Libraries for fields, doubly-linked lists and red-black trees.
   Copyright (C) 2018 James S. Plank */

#ifndef _DLLIST_H_
#define _DLLIST_H_

#include "jval.h"

typedef struct dllist {
    struct dllist *flink;
    struct dllist *blink;
    Jval val;
} *Dllist;

extern Dllist new_dllist();
extern void free_dllist(Dllist);

extern void dll_append(Dllist, Jval);
extern void dll_prepend(Dllist, Jval);
extern void dll_insert_b(Dllist, Jval);
extern void dll_insert_a(Dllist, Jval);

extern void dll_delete_node(Dllist);
extern int dll_empty(Dllist);
```


basic operations - dllist

- **Dllist new_dllist()**: Allocates and returns a new doubly linked list.
- **free_dllist(Dllist l)**: Destroys the list, calling **free()** on all allocated memory in the list. The list does not have to be empty.
- **dll_prepend(Dllist l, Jval val)**: Adds a new node at the beginning of the list. This node's value is **val**. **Dll_prepend()** has no return value.
- **dll_append(Dllist l, Jval val)**: Adds a new node at the end of the list. This node's value is **val**. **Dll_append()** has no return value.
- **dll_insert_b(Dllist n, Jval val)**: Adds a new node to the list right before the specified node. This node's value is **val**.
- **dll_insert_a(Dllist n, Jval val)**: Adds a new node to the list right after the specified node. This node's value is **val**.
- **dll_delete_node(Dllist)**: Deletes the given node from the list. Don't call this on the sentinel.
- **int dll_empty(Dllist l)**: Returns whether **l** is empty.

Other operations

- **Dlist dll_nil(Dlist l)**: Returns a pointer to the **nil** (sentinel) node for the list. You can think of **nil** as a node with no value, that begins and ends the list. Since **l** points to the sentinel node, **dll_nil** returns **l**. You don't need to call **dll_nil()** to access the sentinel node. You can just use **l**, although it makes your code more readable if you use **dll_nil()**.
- **Dlist dll_first(Dlist l)**: Returns a pointer to the first node in the list. If the list is empty, this returns the sentinel. As with **dll_nil()**, you don't need to call **dll_first(l)** -- you can just use **l->flink**.
- **Dlist dll_last(Dlist l)**: Returns a pointer to the last node in the list. If the list is empty, this returns the sentinel. As with **dll_nil()**, you don't need to call **dll_last(l)** -- you can just use **l->blink**.
- **Dlist dll_next(Dlist n)**: Returns a pointer to the next node in the list after **n**. If **n** is the last node on the list, then **dll_next(n)** returns the sentinel. As with **dll_first()**, you don't need to call **dll_next(n)** -- you can just use **n->flink**.
- **Dlist dll_prev(Dlist n)**: Returns a pointer to the previous node in the list before **n**. If **n** is the first node on the list, then **dll_prev(n)** returns the sentinel. As with **dll_last()**, you don't need to call **dll_prev(n)** -- you can just use **n->blink**.
- **Jval dll_val(Dlist n)**: Returns node **n**'s **val** field. Again, you don't need to use this, but sometimes it comes in handy.
- **int dll_delete_node(Dlist n)**: Deletes and frees node **n**.

2 macros

□ Finally, there are two macros for switching back and forth through dll lists.

```
#define dll_traverse ( ptr , list ) \  
    for ( ptr = ( list )->flink ; ptr != ( list ); ptr = ptr -> flink )  
  
#define dll_rtraverse ( ptr , list ) \  
    for ( ptr = ( list )->blink ; ptr != ( list ); ptr = ptr -> blink )
```

Dllist structure

- It's like a circular, doubly linked list.

Some ascii art: Here's an empty list **l**, that would be returned from **new_dllist()**:

```
1  -----> |-----|
      /--->| flink -----\
      | /->| blink -----\ | | | |
      | |  | val = ?      | |
      | |  | -----|    | |
      | |  |             | |
      | \-----|-----/ |
      |-----|-----/
```

And here's that list after calling **dll_append(l, new_jval_i(3));** (or **dll_prepend(l, new_jval_i(3))** for that matter).

```
1  -----> |-----| |-----|
      /--->| flink ----->| flink -----\
      | /->| blink ----->| blink -----\ | | | | |
      | |  | val = ?      |      | val.i = 3 | |
      | |  | -----|    |      | -----| |
      | |  |             |      |             |
      | \-----|-----/ |      |-----/ |
      |-----|-----/
```

Dllist structure

Actually, it makes the drawing cleaner to have the back links go backwards:

```
1  ----->|-----|-----|
    /--->| flink ----->| flink -----\
    | /----- blink |<----- blink |<-\ | | | |
    | | val = ?      |      | val.i = 3 | | |
    | | -----|      | -----| | |
    | | \-----/ |      | | |
    | \-----/ |      | | |
    | \-----/ |      | | |
    \-----/ |      | | |
    \-----/ |      | | |
```

Here's that list after calling **dll_append(l, new_jval_i(5));**:

```
1  ----->|-----|-----|-----|
    /--->| flink ----->| flink ----->| flink -----\
    | /----- blink |<----- blink |<----- blink |<-\ | | | | |
    | | val = ?      |      | val.i = 3 |      | val.i = 5 | | |
    | | -----|      | -----|      | -----| | |
    | | \-----/ |      | | |      | | |
    | \-----/ |      | | |      | | |
    | \-----/ |      | | |      | | |
    \-----/ |      | | |      | | |
    \-----/ |      | | |      | | |
```

src / dllreverse.c

```
/* Printing the lines of standard input in reverse order, by appending
   to a dllist, and then printing the dllist backward. */

#include <stdio.h>
#include <string.h>
#include "fields.h"
#include "dllist.h"

int main()
{
    IS is;
    Dllist l;
    Dllist tmp;

    is = new_inputstruct(NULL);
    l = new_dllist();

    while (get_line(is) >= 0) {
        dll_append(l, new_jval_s(strdup(is->text1)));
    }

    /* Notice how convenient this macro is! */

    dll_rtraverse(tmp, l) printf("%s", tmp->val.s);
    return 0;
}
```

src / dllreverse.c

```
unalc@unalc:~/Desktop/cs360/Dllists/bin$ ./dllreverse < ../makefile
```

```
$(CC) $(CFLAGS) -o bin/dlltail src/dlltail.c $(LIBS)
bin/dlltail: src/dlltail.c
```

```
$(CC) $(CFLAGS) -o bin/cos src/cos.c $(LIBS)
bin/cos: src/cos.c
```

```
$(CC) $(CFLAGS) -o bin/dllreverse src/dllreverse.c $(LIBS)
bin/dllreverse: src/dllreverse.c
```

```
rm -f bin/*
clean:
```

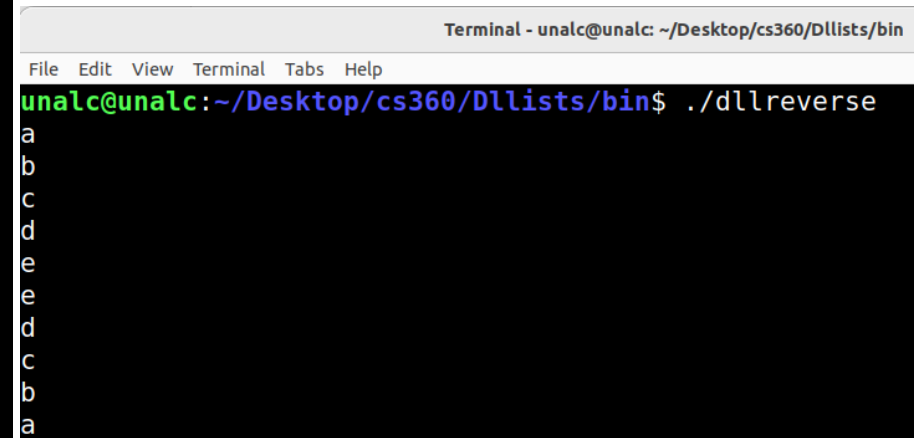
```
all: $(EXECUTABLES)
```

```
CC = gcc
EXECUTABLES = bin/dllreverse bin/dlltail bin/cos
```

```
LIBS = $(CLASSDIR)/lib/libfdr.a
CFLAGS = $(INCLUDE)
INCLUDE = -I$(CLASSDIR)/include
CLASSDIR = ../Libfdr
```

```
# that they point to the correct directories/library.
```

```
# If you want to use this on your own machine, change CLASSDIR and/or INCLUDE and LIBS so
```



The screenshot shows a terminal window titled "Terminal - unalc@unalc: ~/Desktop/cs360/Dllists/bin". The terminal has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The prompt is "unalc@unalc:~/Desktop/cs360/Dllists/bin\$./dllreverse". The output of the command is a vertical list of characters: "a", "b", "c", "d", "e", "e", "d", "c", "b", "a".

src / dlltail.c

- Printing the last n lines of standard input. We do this by reading the standard input to a Dllist and ensuring that the Dllist always has at most n nodes. Then we print it out.

```
/* This program implements tail using a dllist. It reads lines of standard input until it contains the proper number of lines. Then, each time it reads a new line, it appends it to the dllist and then deletes the first line of the dllist. Finally, it prints the dllist. */

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "fields.h"
#include "dllist.h"

int main(int argc, char **argv)
{
    IS is;
    int n;
    Dllist l;
    Dllist tmp;

    /* Error check and read the number of lines to maintain. */

    if (argc != 2) {
        fprintf(stderr, "usage: dlltail n\n");
        exit(1);
    }
    n = atoi(argv[1]);
    if (n < 0) {
        fprintf(stderr, "usage: dlltail n -- n must be >= 0\n");
        exit(1);
    }
}
```

```
/* Create the inputstruct and dllist */

is = new_inputstruct(NULL);
l = new_dllist();

/* Read each line and append to the dllist. */

while (get_line(is) >= 0) {
    dll_append(l, new_jval_s(strdup(is->text1)));

    if (is->line > n) {
        /* If the dllist is full, delete the first node. */
        free(l->flink->val.s);
        dll_delete_node(l->flink);
    }
}

/* At the end, print the dllist -- again note the c */

dll_traverse(tmp, l) printf("%s", tmp->val.s);
return 0;
}
```


src / dlltail.c .

- ❑ A few notes about this program. First, you must call `strdup ()` to copy the string you read from standard input.
- ❑ Otherwise, `get_line ()` will overwrite the string and all your lines will be the same.
- ❑ Second, when you call `dll_delete_node ()`, it removes the node from the list and frees the node.
- ❑ However, if the `val` field shows data allocated with `malloc ()`, then this data is not automatically freed.
- ❑ This is because the `dllist` library has no idea what the data is.
- ❑ Maybe you want him to be released, maybe you don't.

COS: Columns of strings and doubles

- The non-double, padded to 30 characters and left justified.
- A space.
- The double, padded to 20 characters, right justified with four digits after the decimal point. _____

- ❑ This program reads the words and puts each word into one of two lists.
- ❑ the word is double , it puts it into a list. If not, it puts it on the other list.
- ❑ It then prints the two lists formatted in two columns.

```
UNIX> cat txt/input1.txt
1 Fred 2 3 Binky Dontonio
UNIX> bin/cos < txt/input1.txt
Fred                1.0000
Binky               2.0000
Dontonio            3.0000
UNIX> cat txt/input2.txt
1 2 3 Fred
UNIX> cos < txt/input2.txt
Fred                1.0000
                   2.0000
                   3.0000

UNIX> cat txt/input3.txt
1 2 Fred Binky Dontonio
UNIX> bin/cos < txt/input3.txt
Fred                1.0000
Binky               2.0000
Dontonio
UNIX> bin/cos < txt/input3.txt | cat -e
Fred                1.0000$
Binky               2.0000$
Dontonio            $
UNIX>
```

```

/* This program reads words and puts each word onto one of two lists.  If the word is
a double, it puts it on one list.  If it's not, it puts it on the other list.  It
then prints the two lists in two columns, formatted.  Please see the lecture notes
for a precise definition of the format. */

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "fields.h"
#include "dllist.h"

```

```

int main()

```

```

{
    Dllist dl, dtmp; /* The list for doubles */
    Dllist sl, stmp; /* The list for strings */
    double d;
    int i;
    IS is;

```

```

    /* Create the lists and the inputstruct */

```

```

    dl = new_dllist();
    sl = new_dllist();
    is = new_inputstruct(NULL);

```

```

    /* Append to the two lists, using sscanf to differentiate between numbe

```

```

    while (get_line(is) >= 0) {
        for (i = 0; i < is->NF; i++) {
            if (sscanf(is->fields[i], "%lf", &d) == 1) {
                dll_append(dl, new_jval_d(d));
            } else {
                dll_append(sl, new_jval_s(strdup(is->fields[i])));
            }
        }
    }
}

```

```

    /* Print the lines that have both doubles and strings.

```

```

        You'll note -- I'm using the sentinel to test for the end of each list */

```

```

    dtmp = dl->flink;
    stmp = sl->flink;
    while (dtmp != dl && stmp != sl) {
        printf("%-30s %20.4lf\n", stmp->val.s, dtmp->val.d);
        dtmp = dtmp->flink;
        stmp = stmp->flink;
    }

```

```

    /* Now print the lines that have strings only: */

```

```

    while (stmp != sl) {
        printf("%-30s %20s\n", stmp->val.s, "");
        stmp = stmp->flink;
    }

```

```

    /* And print the lines that have doubles only: */

```

```

    while (dtmp != dl) {
        printf("%-30s %20.4lf\n", "", dtmp->val.d);
        dtmp = dtmp->flink;
    }

```

```

    return 0;

```

```

}

```

Application details

```
Dllist new_dllist()
{
    Dllist d;

    d = (Dllist) malloc (sizeof(struct dllist));
    d->flink = d;
    d->blink = d;
    return d;
}
```

```
dll_empty(Dllist l)
{
    return (l->flink == l);
}
```

```
free_dllist(Dllist l)
{
    while (!dll_empty(l)) {
        dll_delete_node(dll_first(l));
    }
    free(l);
}
```

```
free_dllist(Dllist l)
{
    while (!dll_empty(l)) {
        dll_delete_node(dll_first(l));
    }
    free(l);
}
```

```
dll_insert_b(Dllist node, Jval v)    /* Inserts before a given node */
{
    Dllist new;

    new = (Dllist) malloc (sizeof(struct dllist));
    new->val = v;

    new->flink = node;
    new->blink = node->blink;
    new->flink->blink = new;
    new->blink->flink = new;
}
```

Application details

```
dll_insert_a(Dllist n, Jval val)      /* Inserts after a given node */
{
    dll_insert_b(n->flink, val);
}

dll_append(Dllist l, Jval val)        /* Inserts at the end of the list */
{
    dll_insert_b(l, val);
}

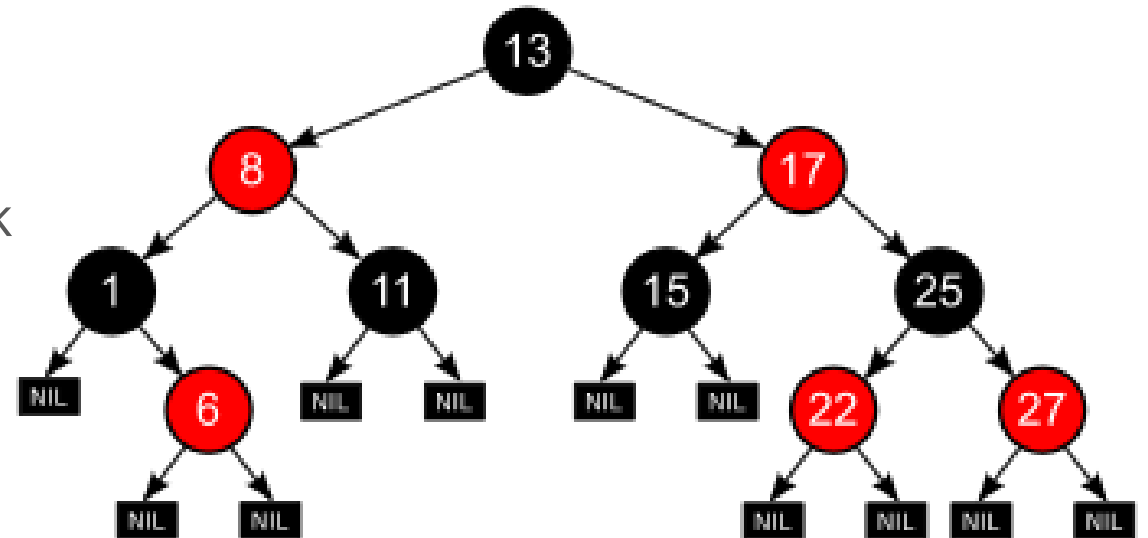
dll_prepend(Dllist l, Jval val)       /* Inserts at the beginning of the list */
{
    dll_insert_b(l->flink, val);
}

free_dllist dll_delete_node(Dllist node) /* Deletes an arbitrary item */
{
    while (!d {
        node->flink->blink = node->blink;
        dll_del node->blink->flink = node->flink;
    }
    free(l); free(node);
}
```

Red -Black Trees

- ❑ Rb-trees are data structures based on balanced binary trees.
- ❑ You don't need to know how they work -- just know that they work and that all operations occur in $O(\log(n))$ time, where n is the number of elements in the tree.

1. Each node , each red or black colourful two child.
2. Every tree leaf node black colorful .
3. Every red both of the node his child is also black colorful .
4. Radical One tree to the leaf Every road is the same number (" black height ") black loop includes



Red -Black Trees

- ❑ **JRB jrb_insert_str (JRB tree, char * key , Jval val):** add a new node to the tree using a standard character string as the key.
- ❑ **JRB jrb_insert_int (JRB tree, int key, Jval val):** add to the tree using an integer as the key.
- ❑ **JRB jrb_insert_dbl (JRB tree, double key, Jval val):** **Add it to the tree using** double as the key.
- ❑ **JRB jrb_insert_gen (JRB tree, Jval key, Jval val , int (* func)(Jval , Jval))** Now, your key is a jval
You provide a comparison function func () that takes two Jvals as arguments and returns :
 - ❑ A negative integer if the first key is less than the second.
 - ❑ A positive integer if the first key is greater than the second.
 - ❑ zero if the keys are equal.

Examples

Example programs:

- [src/strsort.c](#): Uses red-black trees to sort standard input lexicographically.
- [src/strrsort1.c](#): Uses red-black trees to sort standard input lexicographically in reverse order. It does this by traversing the tree in reverse order.
- [src/strrsort2.c](#): Uses red-black trees to sort standard input lexicographically in reverse order. It does this by creating a new comparison function **revcomp**, which simply returns `-strcmp()`. Now the tree sorts in reverse order, so it is traversed in the forward direction.
- [src/strusort.c](#): Uses red-black trees to sort standard input lexicographically, and it removes duplicate lines. It does this by checking for a line before inserting it into the tree.
- [src/strisort.c](#): Uses red-black trees to sort standard input lexicographically, ignoring upper and lower case. It does this by creating a separate key, which is the string in upper case. It sorts with that string, and stores the original string in the **val**.
- [src/nsort.c](#): Uses red-black trees to sort like **sort -n --** i.e. it treats each line as an integer, and sorts it that way. If the lines are not integers, or there are duplicate lines, anything goes.
- [src/nsort2.c](#): Uses red-black trees to sort like **sort -n** only now if two lines have the same **atoi()** value, then they are sorted lexicographically. This uses **jrb_insert_gen()**.
- [src/nsort3.c](#): Same as **nsort2**, but instead it uses a two-level red-black tree. If this is confusing to you, please read the next example, which does a very similar thing.

Assignments

- * Evaluate the size of an element from struct in fields.h and union in jval.h.
 - * Using libfdr libraries, write a program that finds the number of words in a txt file given as a parameter.
-

Example input file content:

The Canadian Institute for Cybersecurity (CIC) is a prominent institution with a track record of impactful contributions to cybersecurity. Through dataset provision, industry partnerships, and an IoT lab, they actively support research and initiatives to advance IoT security.

Runnning example C:\Desktop\cs360> ./test <input.txt

```
*****  
***
```

- * Using libfdr libraries, save the numeric values in a txt file given as a parameter to a dll structure. When the reading process is finished, print the numeric values in reverse order on the screen.

Example input file content:

The Canadian 55 Institute 75 for Cybersecurity (CIC) is a prominent 43 institution with a track 32 record of impactful contributions to cybersecurity 1000. Through dataset provision, industry 87 partnerships, and an 21 IoT lab, they actively support research 98 and initiatives to advance IoT security.

Runnning example C:\Desktop\cs360> ./test <input.txt